



Sume parțiale | Șmenul lui Mars

Tablouri unidimensionale · Tablouri bidimensionale

Andrei Mihai Ciobanu

Algolymp Summer School

Cuprins

Sume parțiale pe vectori

Sume parțiale pe matrici

Șmenul lui Mars pe vector

Șmenul lui Mars pe matrici

Recapitulare

Resurse

Probleme de antrenament

Sume parțiale pe vectori

Problema

Avem un vector $a[1 \dots n]$ și Q interogări de tipul:

Interogare

Care este suma $a[l] + a[l + 1] + \dots + a[r]$?

Soluție naivă: parcurgem elementele din $[l, r]$ și le adunăm.

- O interogare costă $O(n)$ — Q interogări costă $O(n \cdot Q)$
- Pentru $n = Q = 10^5$: 10^{10} operații — prea lent!

Idee: facem o precalculare în $O(n)$ care ne permite să răspundem la fiecare interogare în $O(1)$.

Abordare	Precalcul	Per interogare
Naivă	$O(1)$	$O(n)$
Sume parțiale	$O(n)$	$O(1)$

Ce stochează $S[i]$?

Construim un vector auxiliar $S[0 \dots n]$ cu proprietatea:

$S[i]$ = suma primelor i elemente din a ($S[0] = 0$ prin convenție)

Exemplu cu $a = [3, 1, 4, 1, 5, 9]$:

i	0	1	2	3	4	5	6
$a[i]$	—	3	1	4	1	5	9
$S[i]$	0	3	4	8	9	14	23

$$S[3] = a[1] + a[2] + a[3] = 3 + 1 + 4 = 8$$

$$S[5] = a[1] + \dots + a[5] = 14$$

Construcție: pornim cu $S[0] = 0$, apoi $S[i] = S[i - 1] + a[i]$

Cum aflăm suma unui segment?

Interogare: $\text{sum}(l, r) = S[r] - S[l - 1]$

De ce funcționează? Scriem explicit ce conține fiecare termen:

$$S[r] = a[1] + a[2] + \dots + a[l - 1] + a[l] + \dots + a[r]$$

$$S[l - 1] = a[1] + a[2] + \dots + a[l - 1]$$

$$S[r] - S[l - 1] = a[l] + a[l + 1] + \dots + a[r] \quad \checkmark$$

Exemplu: $\text{sum}(3, 5)$ pe $a = [3, 1, 4, 1, 5, 9]$:

i	0	1	2	3	4	5	6
$a[i]$	—	3	1	4	1	5	9
$S[i]$	0	3	4	8	9	14	23

$$\text{sum}(3, 5) = S[5] - S[2] = 14 - 4 = 10 \quad (4 + 1 + 5 = 10 \quad \checkmark)$$

Implementare

```
1  const int nmax = 100000;  
2  int a[nmax + 1];  
3  long long S[nmax + 1];  
4  int n, q, l, r;  
5  
6  // precalcul:  $O(n)$   
7  //  $S[0] = 0$  automat (declarare globala)  
8  for (int i = 1; i <= n; i++) {  
9      S[i] = S[i - 1] + a[i];  
10 }  
11  
12 // interogare  $sum(l, r): O(1)$   
13 long long suma = S[r] - S[l - 1];
```

- **Atenție la overflow!** Dacă $a[i]$ pot fi mari, $S[i]$ poate depăși `int`

Sume parțiale pe matrici

De la 1D la 2D

Aceeași idee, extinsă la matrici; definim:

$S[i][j]$ = suma tuturor elementelor din submatricea cu colțul stânga-sus la $(1, 1)$ și colțul dreapta-jos la (i, j)

Matricea a :

1	2	3
4	5	6
7	8	9

Matricea S :

1	3	6
5	12	21
12	27	45

$$S[2][3] = 1 + 2 + 3 + 4 + 5 + 6 = 21$$

$$S[3][3] = 1 + 2 + \dots + 9 = 45$$

Calculăm S rând cu rând, coloană cu coloană. La momentul calculului lui $S[i][j]$, valorile de deasupra și din stânga sunt deja cunoscute:

Ce avem calculat în jurul lui $S[i][j]$?

- $S[i-1][j]$ — suma dreptunghiului de deasupra
- $S[i][j-1]$ — suma dreptunghiului din stânga
- $S[i-1][j-1]$ — suma dreptunghiului din colțul stânga-sus

Cum calculăm $S[i][j]$?

- Adunăm $S[i-1][j]$ și $S[i][j-1]$ — zona deasupra + zona din stânga
- $S[i-1][j-1]$ a fost adăugat de două ori $\Rightarrow$ îl scădem
- Adăugăm $a[i][j]$ — celula curentă

$$S[i][j] = S[i-1][j] + S[i][j-1] - S[i-1][j-1] + a[i][j]$$

De ce funcționează formula?

Exemplu: calculul lui $S[3][3]$ pe o matrice 3×3 . Numerele arată de câte ori a fost adăugată fiecare celulă:

$+S[2][3]$

1	1	1
1	1	1
0	0	0

$+S[3][2]$

2	2	1
2	2	1
1	1	0

$-S[2][2]$

1	1	1
1	1	1
1	1	0

$+a[3][3]$

1	1	1
1	1	1
1	1	1

Fiecare celulă apare exact o dată $\Rightarrow S[3][3] = \text{suma celor 9 elemente} \checkmark$

Interogarea

Vrem suma elementelor din submatricea cu colțul stânga-sus (r_1, c_1) și dreapta-jos (r_2, c_2) :

Cum? Construim răspunsul din valorile lui S deja calculate:

- Pornim cu $S[r_2][c_2]$ — include tot de la $(1, 1)$ la (r_2, c_2)
- Scădem $S[r_1-1][c_2]$ — eliminăm banda de deasupra zonei dorite
- Scădem $S[r_2][c_1-1]$ — eliminăm banda din stânga zonei dorite
- Adunăm $S[r_1-1][c_1-1]$ — colțul stânga-sus a fost scăzut de două ori, îl readăugăm

$$\begin{aligned} \text{sum}(r_1, c_1, r_2, c_2) = & S[r_2][c_2] - S[r_1-1][c_2] \\ & - S[r_2][c_1-1] + S[r_1-1][c_1-1] \end{aligned}$$

De ce funcționează interogarea?

Exemplu: suma submatricei $(2,2) \dots (4,4)$ pe o matrice 4×4 :

$+S[4][4]$

1	1	1	1
1	1	1	1
1	1	1	1
1	1	1	1

$-S[1][4]$

0	0	0	0
1	1	1	1
1	1	1	1
1	1	1	1

$-S[4][1]$

-1	0	0	0
0	1	1	1
0	1	1	1
0	1	1	1

$+S[1][1]$

0	0	0	0
0	1	1	1
0	1	1	1
0	1	1	1

Zona $(2,2) \dots (4,4)$ — toate celulele cu valoarea 1 $\Rightarrow$ suma corectă ✓

Implementare

```
1  const int nmax = 1000;
2  int a[nmax + 1][nmax + 1];
3  long long S[nmax + 1][nmax + 1];
4
5  // precalculare:  $O(n * m)$ 
6  for (int i = 1; i <= n; i++) {
7      for (int j = 1; j <= m; j++) {
8          S[i][j] = S[i - 1][j] + S[i][j - 1]
9                  - S[i - 1][j - 1] + a[i][j];
10     }
11 }
12
13 // interogare  $(r1, c1) \dots (r2, c2)$ :  $O(1)$ 
14 long long suma = S[r2][c2] - S[r1 - 1][c2]
15               - S[r2][c1 - 1] + S[r1 - 1][c1 - 1];
```

- **Atenție:** de cele mai multe ori suma valorilor matricii poate deveni foarte mare, în consecință declarăm S ca long long

Şmenul lui Mars pe vector

Problema

Avem un vector $a[1 \dots n]$ și Q operații:

Update

Adăugă valoarea v la toate elementele $a[l], \dots, a[r]$

- **Soluție naivă:** parcurgem $[l, r]$ și adăugăm v — $O(n)$ per update
- **Șmenul lui Mars:** fiecare update în $O(1)$, reconstituire finală în $O(n)$

Tehnică	Update pe interval	Citire element
Naiv	$O(n)$	$O(1)$
Șmenul lui Mars	$O(1)$	$O(n)$ reconstituire

Optim când facem **toate update-urile mai întâi**, apoi citim rezultatul.

Vectorul de diferențe

Definiție

$D[i] = a[i] - a[i - 1]$, cu $a[0] = 0$

Update $[l, r]$ cu $+v$: doar două operații!

- $D[l] += v$
- $D[r + 1] -= v$

Exemplu: adăugăm $+3$ pe $[2, 4]$ în $a = [0, 0, 0, 0, 0]$:

	1	2	3	4	5
D	0	+3	0	0	-3
a	0	3	3	3	0

Observație: D e vectorul de diferențe al lui $a \iff$ prin sume parțiale pe D obținem a

Implementare

```
1  const int nmax = 100000;
2  int D[nmax + 2]; // vectorul de diferente
3  int n, q;
4
5  // update [l, r] cu +v: O(1)
6  D[l] += v;
7  D[r + 1] -= v;
8
9  // reconstruire finala: O(n)
10 for (int i = 1; i <= n; i++) {
11     D[i] += D[i - 1]; // D[i] devine a[i]
12 }
```

- D trebuie dimensionat cu $n + 1$ pentru a evita depășirea la $D[r + 1]$
- Reconstituirea se face o singură dată, **după** toate update-urile

Şmenul lui Mars pe matrici

Extensia la matrici

Update pe submatricea $(r_1, c_1) \dots (r_2, c_2)$ cu $+v$ — 4 modificări în colțuri:

Cele 4 actualizări

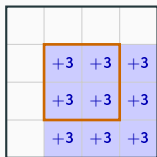
- $D[r_1][c_1] += v$
- $D[r_1][c_2+1] -= v$
- $D[r_2+1][c_1] -= v$
- $D[r_2+1][c_2+1] += v$

Reconstituire: sumă parțială 2D aplicată pe D (aceeași formulă ca la sume parțiale 2D).

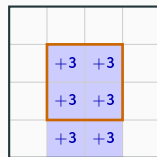
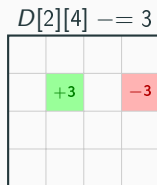
$+v$			$-v$
$-v$			$+v$

De ce funcționează?

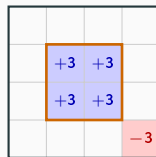
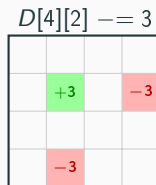
Exemplu: $+3$ pe $(2,2) \dots (3,3)$ în matricea 4×4 zero. Urmărim D și a reconstituită după fiecare colț adăugat:



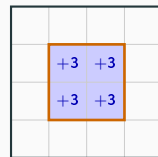
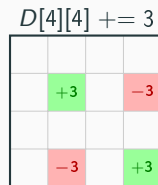
a



a



a



a

Implementare

```
1  const int nmax = 1000;
2  long long D[nmax + 2][nmax + 2];
3
4  // update (r1,c1)..(r2,c2) cu +v: O(1)
5  void update(int r1, int c1, int r2, int c2,
6              long long v) {
7      D[r1][c1] += v;
8      D[r1][c2 + 1] -= v;
9      D[r2 + 1][c1] -= v;
10     D[r2 + 1][c2 + 1] += v;
11 }
12
13 // reconstituire finala: O(n*m)
14 for (int i = 1; i <= n; i++) {
15     for (int j = 1; j <= m; j++) {
16         D[i][j] += D[i-1][j] + D[i][j-1] - D[i-1][j-1];
17     }
18 }
```

Recapitulare

Recapitulare

Tehnică	Precalcul	Interogare	Update interval
Sume parțiale 1D	$O(n)$	$O(1)$	$O(n)$
Sume parțiale 2D	$O(nm)$	$O(1)$	$O(nm)$
Șmenul lui Mars 1D	$O(n)$	$O(n)$	$O(1)$
Șmenul lui Mars 2D	$O(nm)$	$O(nm)$	$O(1)$

- **Sume parțiale:** multe interogări, vectorul nu se modifică
- **Șmenul lui Mars:** multe update-uri pe intervale, citire o singură dată la final
- Cele două tehnici sunt **inverse** una față de cealaltă

Resurse

- **pbinfo.ro** — Sume parțiale (RO)
- **pbinfo.ro** — Șmenul lui Mars (RO)
- **USACO Guide** — Introduction to Prefix Sums (EN)
- **USACO Guide** — More on Prefix Sums (EN)

Probleme de antrenament

1D

- **Constant-Time Range Sums** (AlgoCode #4203)
- **expresie** (AlgoCode #1835 · OJI 2009 IX)
- **dominant** (AlgoCode #1866 · OJI 2015 VIII)
- **Macarie** (AlgoCode #2984 · OJI 2024 IX)

2D

- **Matrix Line and Diagonal Sums** (AlgoCode #4216)
- **pseudobil** (AlgoCode #1854 · OJI 2014 IX)
- **deminare** (AlgoCode #1905 · OJI 2019 IX)

1D

- **Difference Array Final Values** (AlgoCode #4340)
- **tv** (AlgoCode #2287 · ONI 2015 VII)

2D

- **Rectangle Paint with Difference Arrays** (AlgoCode #4217)
- **drept** (Kilonova #1399 · ONI 2012 X)
- **Eras** (AlgoCode #3077 · ONI 2024 IX)